

A Arquitetura da World Wide Web

Identificação de Recursos (URIs), Interação (Protocolo HTTP) e Representação

Objetivos de Aprendizagem

- Compreender os **princípios base** da arquitetura da World Wide Web.
- Identificar e estruturar **URIs** para recursos web.
- Descrever o funcionamento do **protocolo HTTP** e os seus componentes.
- Reconhecer os **formatos de representação** de recursos (HTML, JSON, XML).

Arquitetura da World Wide Web

- **Modelo Cliente-Servidor**
 - **Cliente:** Navegador web (ex: Chrome, Firefox) que faz pedidos.
 - **Servidor:** Responde com recursos (ex: Apache, Nginx).
- **Comunicação:** Baseada no **protocolo HTTP/HTTPS**.
- **Recursos:** Identificados por **URIs** (ex: `https://www.example.com/index.html`).

Fluxo típico:

1. Cliente envia um **pedido HTTP** (ex: GET `/index.html`).
2. Servidor processa e envia uma **resposta HTTP** (ex: 200 OK + conteúdo HTML).

Identificação de Recursos: URIs

- **URI (Uniform Resource Identifier):** String que identifica um recurso na Web.
- **Estrutura de um URI:**

```
esquema://autoridade/caminho?query#fragmento
```

- **Esquema:** Protocolo (ex: `https` , `http` , `ftp`).
- **Autoridade:** Domínio (ex: `www.example.com:8080`).
- **Caminho:** Localização do recurso (ex: `/path/to/resource`).
- **Query:** Parâmetros (ex: `?id=123&name=test`).
- **Fragmento:** Secção específica (ex: `#section1`).

Exemplo:

```
https://www.example.com:8080/path/to/resource?id=123#section1
```

URIs: URL vs URN

Tipo	Descrição	Exemplo
URL	Localização do recurso (endereço web).	<code>https://www.example.com/index.html</code>
URN	Nome único do recurso (independente de localização).	<code>urn:isbn:0451450523</code>

Nota: Todos os URLs são URIs, mas nem todos os URIs são URLs.

Protocolo HTTP: Visão Geral

- **HTTP (Hypertext Transfer Protocol):** Protocolo de comunicação para transferência de dados na Web.
- **Características:**
 - **Stateless:** Cada pedido é independente (sem memória de pedidos anteriores).
 - **Baseado em texto:** Pedidos e respostas são legíveis por humanos.
 - **Versões:** HTTP/1.1 (RFC 2616), HTTP/2, HTTP/3.

Fluxo HTTP:

1. Cliente abre uma **ligação TCP** ao servidor.
2. Envia um **pedido HTTP** (ex: GET, POST).
3. Servidor processa e envia uma **resposta HTTP**.
4. Ligação é **fechada** (em HTTP/1.1, pode ser mantida com `Connection: keep-alive`).

Métodos HTTP

Método	Descrição	Idempotente	Exemplo de Uso
GET	Obter um recurso.	Sim	<code>GET /index.html</code>
POST	Enviar dados para criar um recurso.	Não	<code>POST /users</code> (com body JSON)
PUT	Atualizar ou substituir um recurso.	Sim	<code>PUT /users/1</code> (com body JSON)
DELETE	Remover um recurso.	Sim	<code>DELETE /users/1</code>
HEAD	Obter headers de um recurso (sem body).	Sim	<code>HEAD /index.html</code>

Códigos de Estado HTTP

Classe	Descrição	Exemplos
1xx	Informacional (pedido recebido).	100 Continue
2xx	Sucesso.	200 OK , 201 Created
3xx	Redirecionamento.	301 Moved Permanently , 302 Found
4xx	Erro do cliente.	404 Not Found , 400 Bad Request
5xx	Erro do servidor.	500 Internal Server Error

Headers HTTP

- Headers de Pedido:

- `Host` : Domínio do servidor (ex: `Host: www.example.com`).
- `User-Agent` : Identificação do cliente (ex: `User-Agent: Mozilla/5.0`).
- `Accept` : Formatos aceites (ex: `Accept: application/json`).
- `Authorization` : Credenciais de autenticação.

- Headers de Resposta:

- `Content-Type` : Formato do conteúdo (ex: `Content-Type: text/html`).
- `Content-Length` : Tamanho do body em bytes.
- `Cache-Control` : Diretivas de cache.

Exemplo de Pedido HTTP:

Representação de Recursos

- **Formato de Representação:** Como o recurso é apresentado ao cliente.
- **Formatos comuns:**
 - **HTML:** Estrutura de páginas web.
 - **JSON:** Dados estruturados (usado em APIs).
 - **XML:** Dados estruturados (alternativa a JSON).

Negociação de Conteúdo:

- Cliente indica formatos aceites via header `Accept` .
- Servidor responde com o formato mais adequado (header `Content-Type`).

Exemplo de Resposta HTTP com JSON:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

Exemplo Prático: Pedido e Resposta HTTP

Pedido (GET):

```
GET /api/users/1 HTTP/1.1  
Host: api.example.com  
Accept: application/json
```

Resposta (200 OK):

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Content-Length: 42  
  
{  
  "id": 1,  
  "name": "Jorge Silva",  
  "email": "jorge@example.com"  
}
```

Síntese: Pontos-Chave

- **Arquitetura Web:** Modelo **cliente-servidor** com comunicação via **HTTP/HTTPS**.
- **URIs:** Identificam recursos (URLs para localização, URNs para nomeação).
- **HTTP:**
 - **Métodos:** GET, POST, PUT, DELETE, etc.
 - **Códigos de estado:** 1xx (info), 2xx (sucesso), 3xx (redireção), 4xx (erro cliente), 5xx (erro servidor).
 - **Headers:** Host , Content-Type , Accept , etc.
- **Representação:** Recursos podem ser representados em **HTML, JSON** ou **XML**.

Próximos Passos: Explorar **REST APIs** e programação cliente/servidor.

Referências

- W3C Technical Architecture Group, *Architecture of the World Wide Web, Volume One* (2004).
- RFC 2616: *Hypertext Transfer Protocol -- HTTP/1.1*.
- *[FONTE EXTERNA]*: Exemplos práticos de HTTP/2 e HTTP/3.